21bcsa92-nikhil-khatri-assign-5

July 7, 2023

[136]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_absolute_error, mean_squared_error
```

1 Assignment Problems: Day 5

The following problems require access to the file named 'Dataset_Day5.csv', provided with this assignment. (Use Random State = 42)

Boston house prices dataset

Data Set Characteristics:

:Number of Instances: 506

:Number of Attributes: 13 numeric/categorical predictive. Median value (attribute 14) is usually the target.

:Attribute Information (in order):

- CRIM per capita crime rate by town
- ZN proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS proportion of non-retail business acres per town
- CHAS Charles River dummy variable (= 1 if tract bounds river; otherwise)
- NOX nitric oxides concentration (parts per 10 million)
- RM - average number of rooms per dwelling
- AGE proportion of owner-occupied units built prior to 1940
- DIS weighted distances to five Boston employment centres
- RAD index of accessibility to radial highways
- TAX full-value property-tax rate per 10,000
- PTRATIO pupil-teacher ratio by town
- B 1000(Bk 0.63)^2 where Bk is the proportion of black people by town
- LSTAT % lower status of the population
- MEDV Median value of owner-occupied homes in $1000's

[151]:
```
data = pd.read_csv("Dataset_Day5.csv")
data["TAX"] = data["TAX"]/10
```

2 question - 1

- Perform a complete data inspection including - (20 marks)
- Missing Data Treatment
- Descriptive Statistics of each variable (Eg. Boxplot, Histogram etc.)
- Visualization of all continuous variables

[152]:
```
data.info(), data.describe(), data.head()
data
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
```

[152]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	\
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	29.6	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	24.2	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	24.2	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	22.2	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	22.2	
..	…	…	…	…	…	…	…	…	…	…	
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1.0	27.3	
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1.0	27.3	
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1.0	27.3	
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1.0	27.3	
505	0.04741	0.0	11.93	0.0	0.573	6.030	80.8	2.5050	1.0	27.3	

```
     PTRATIO       B  LSTAT  MEDV
0       15.3  396.90   4.98  24.0
1       17.8  396.90   9.14  21.6
2       17.8  392.83   4.03  34.7
3       18.7  394.63   2.94  33.4
4       18.7  396.90   5.33  36.2
..       …     …    …   …
501     21.0  391.99   9.67  22.4
502     21.0  396.90   9.08  20.6
503     21.0  396.90   5.64  23.9
504     21.0  393.45   6.48  22.0
505     21.0  396.90   7.88  11.9

[506 rows x 14 columns]
```

[46]:
```
sns.distplot(data.MEDV)
```

```
C:\Users\nikhi\AppData\Local\Temp\ipykernel_9324\1093415441.py:1: UserWarning:

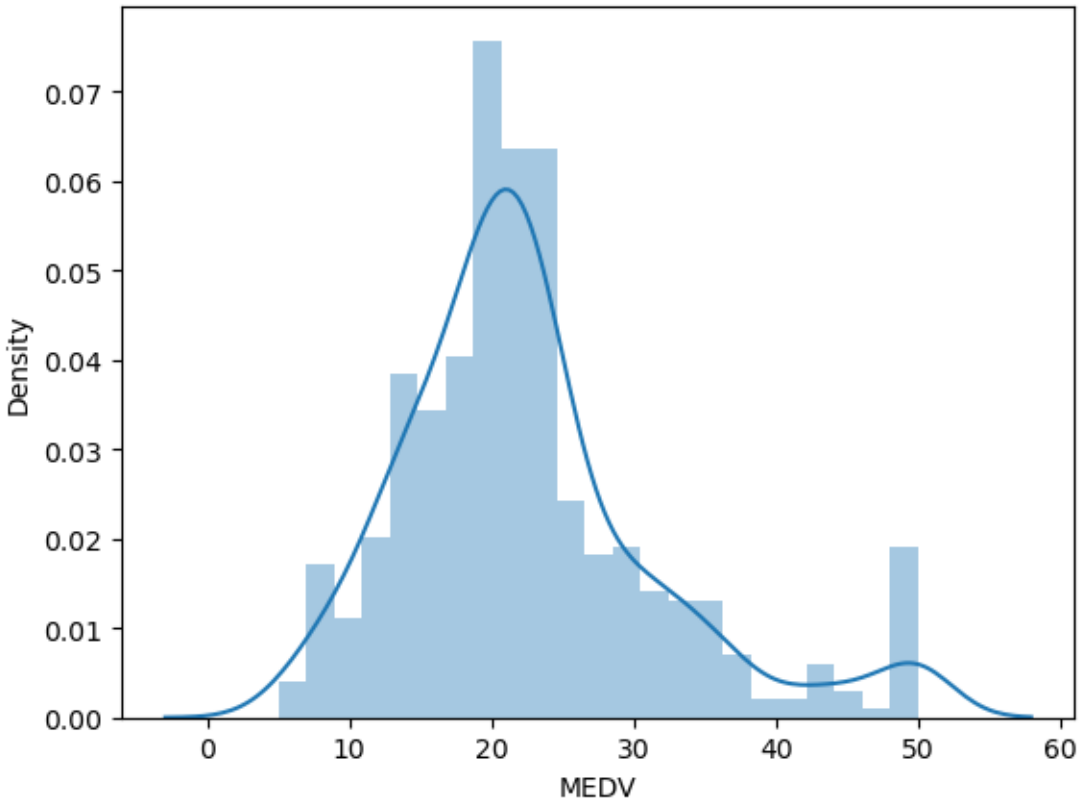
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

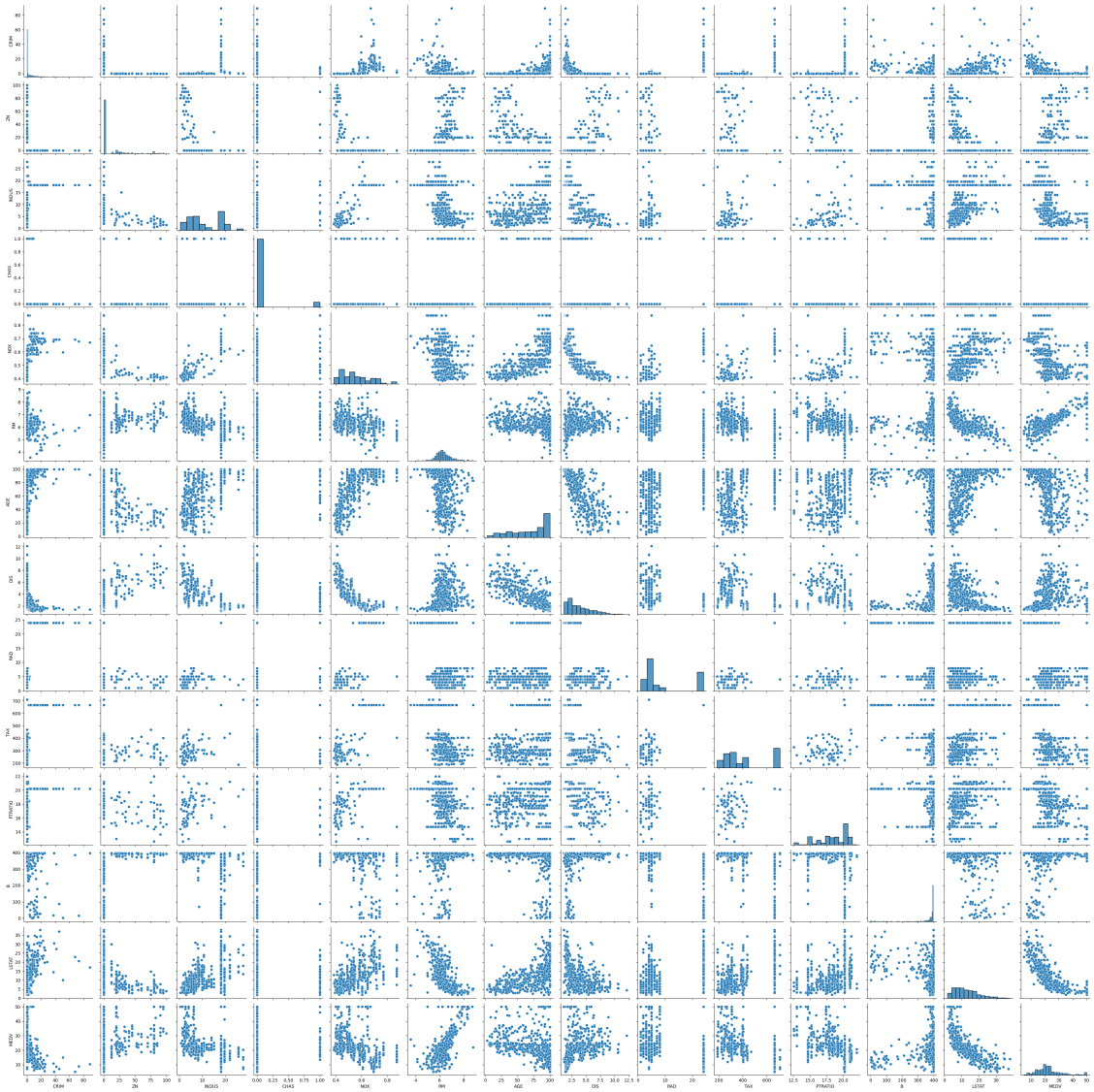
  sns.distplot(data.MEDV)
```

[46]:
```
<Axes: xlabel='MEDV', ylabel='Density'>
```



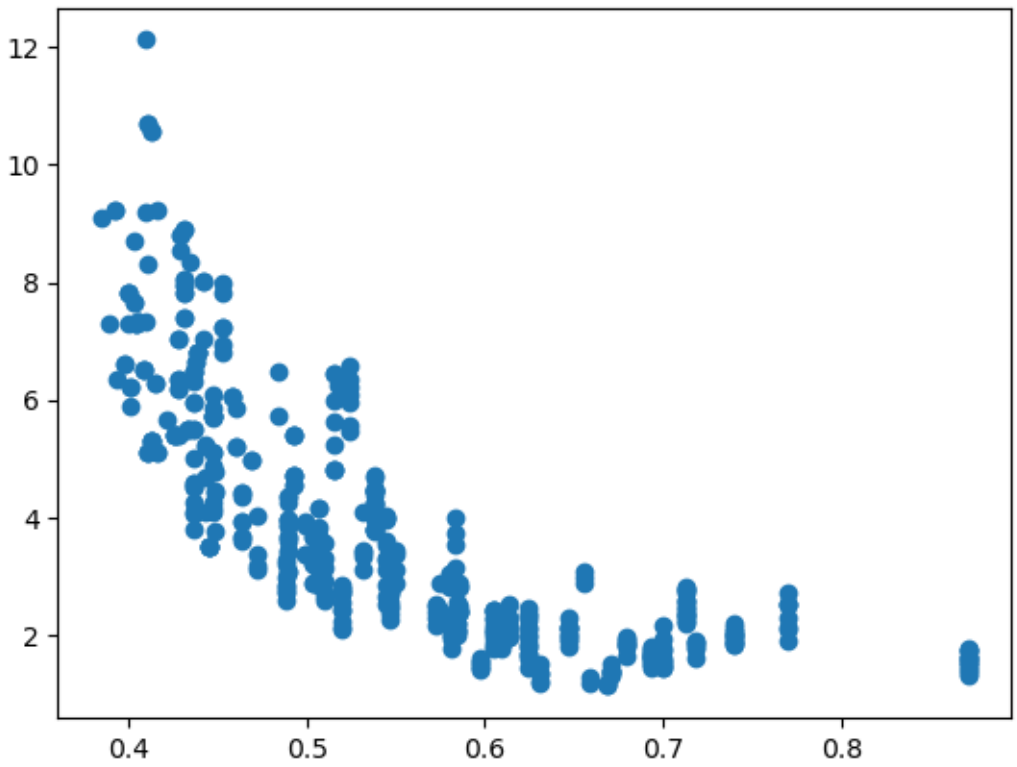
[15]:
```
# from data.info table we conclude that we don't have any null value and domn't need any missing data treatment.
sns.pairplot(data)
```

[15]:
```
<seaborn.axisgrid.PairGrid at 0x1faa598c390>
```



[26]:
```
plt.scatter(data.NOX,data.DIS)
```

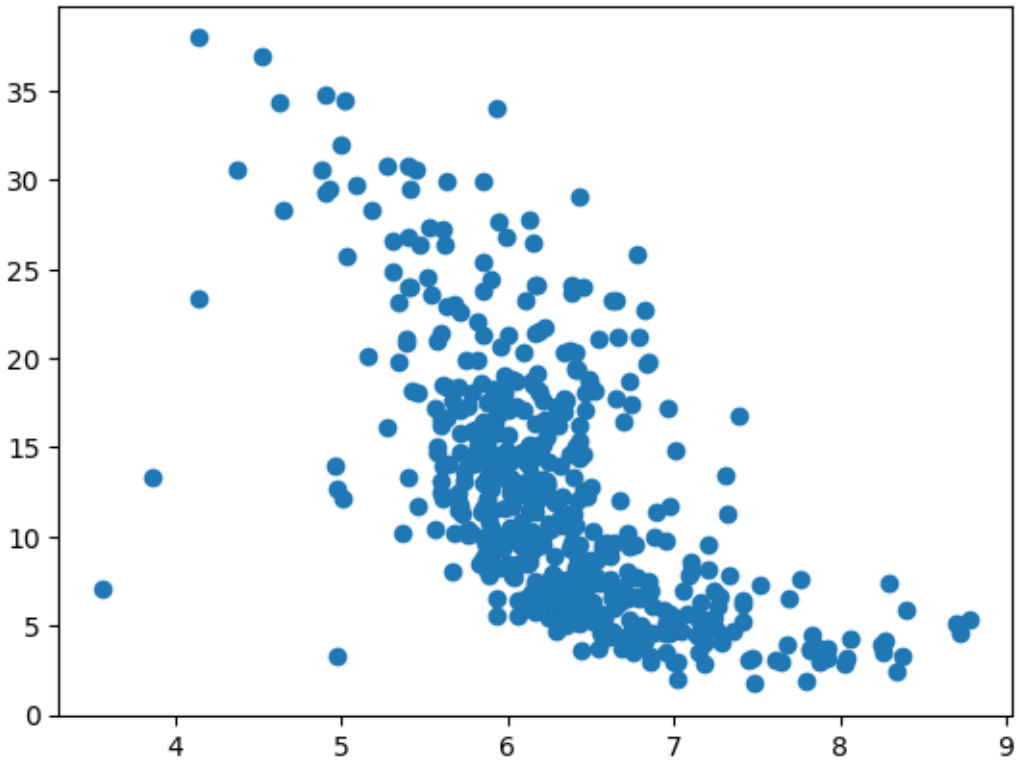
[26]: <matplotlib.collections.PathCollection at 0x1fab2cb0f10>



We can observe a pattern of linear regression graph which almost looks like logarithmic relation

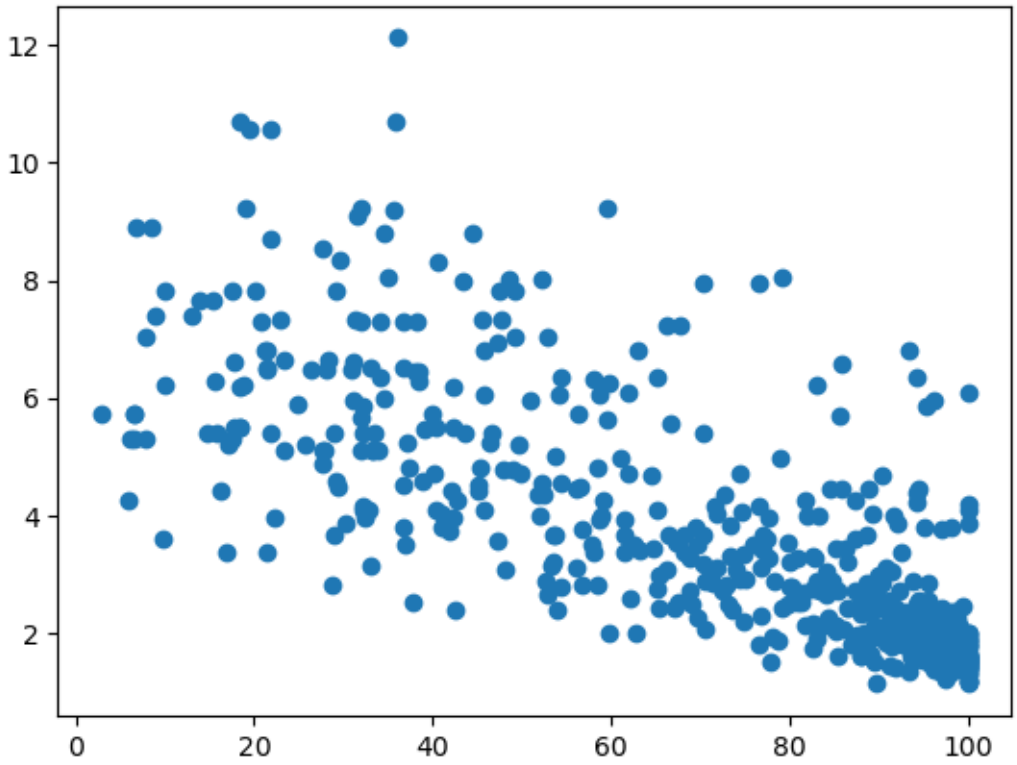
[30]:
```
plt.scatter(data.RM,data.LSTAT)
```

[30]:
```
<matplotlib.collections.PathCollection at 0x1fab2e2cc10>
```



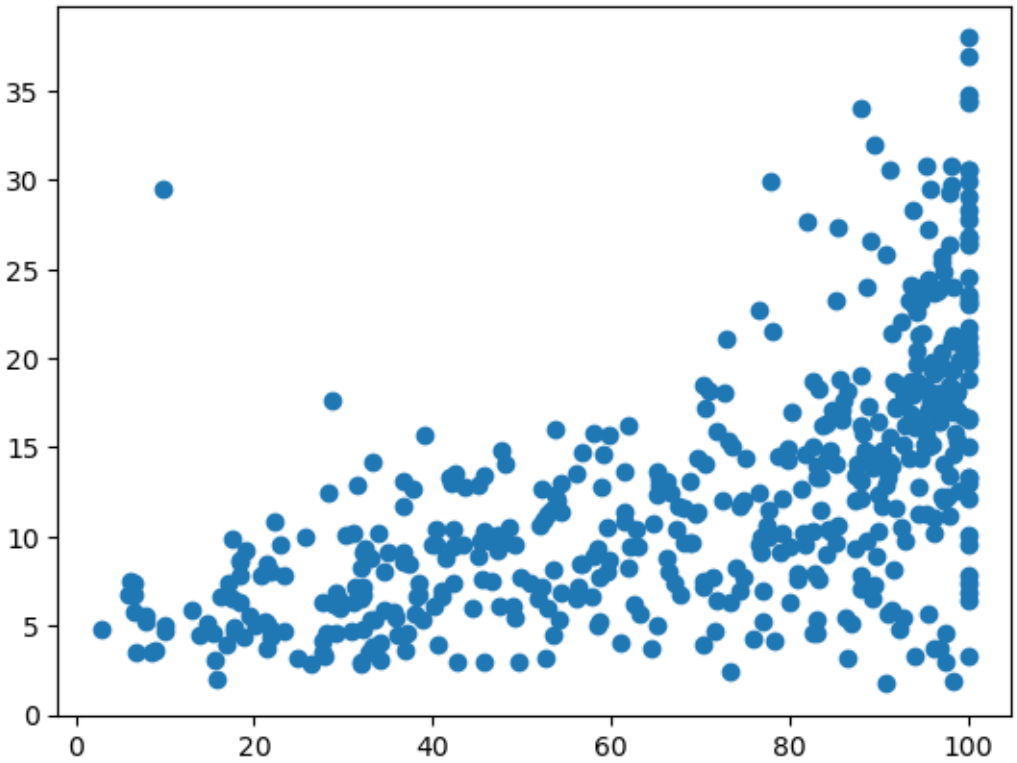
[32]:
```
plt.scatter(data.AGE,data.DIS)
```

[32]: <matplotlib.collections.PathCollection at 0x1fab2da3690>



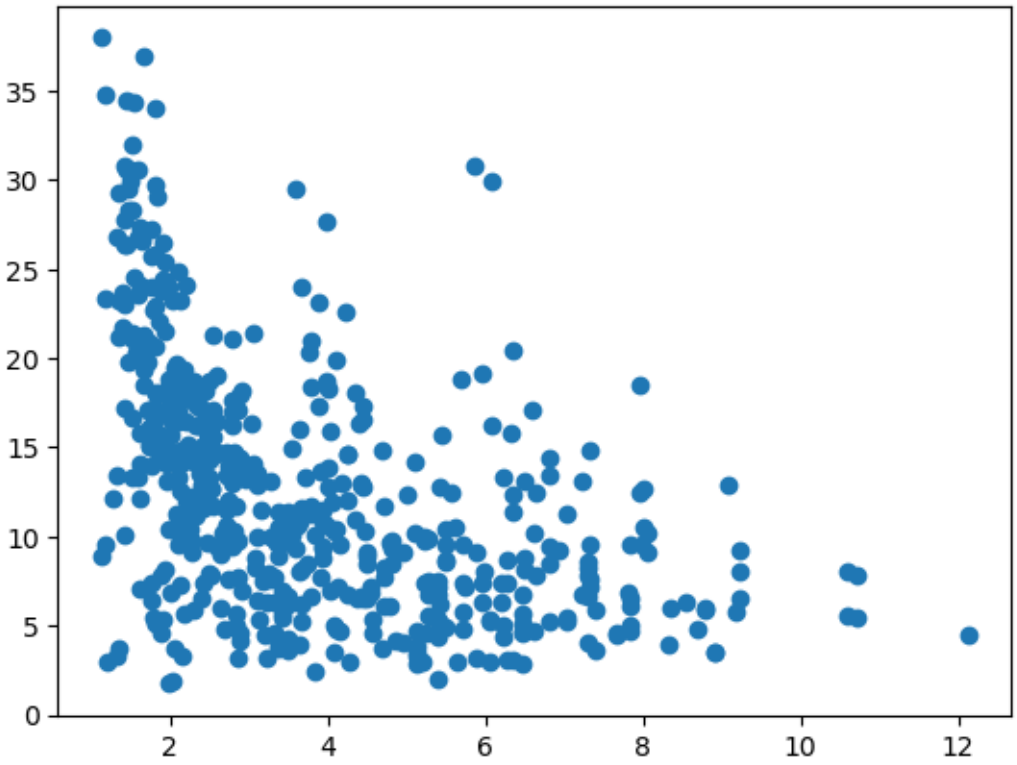
[33]:
```
plt.scatter(data.AGE,data.LSTAT)
```

[33]:
```
<matplotlib.collections.PathCollection at 0x1fab40632d0>
```



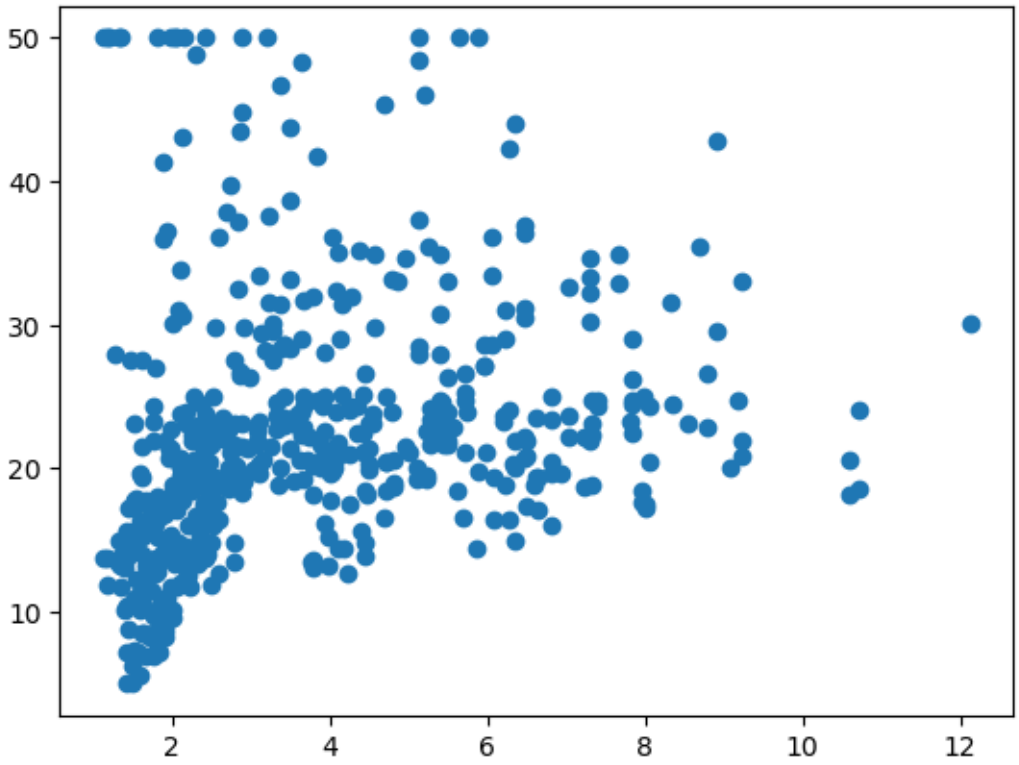
[34]:
```
plt.scatter(data.DIS,data.LSTAT)
```

[34]:
```
<matplotlib.collections.PathCollection at 0x1fab40d9650>
```



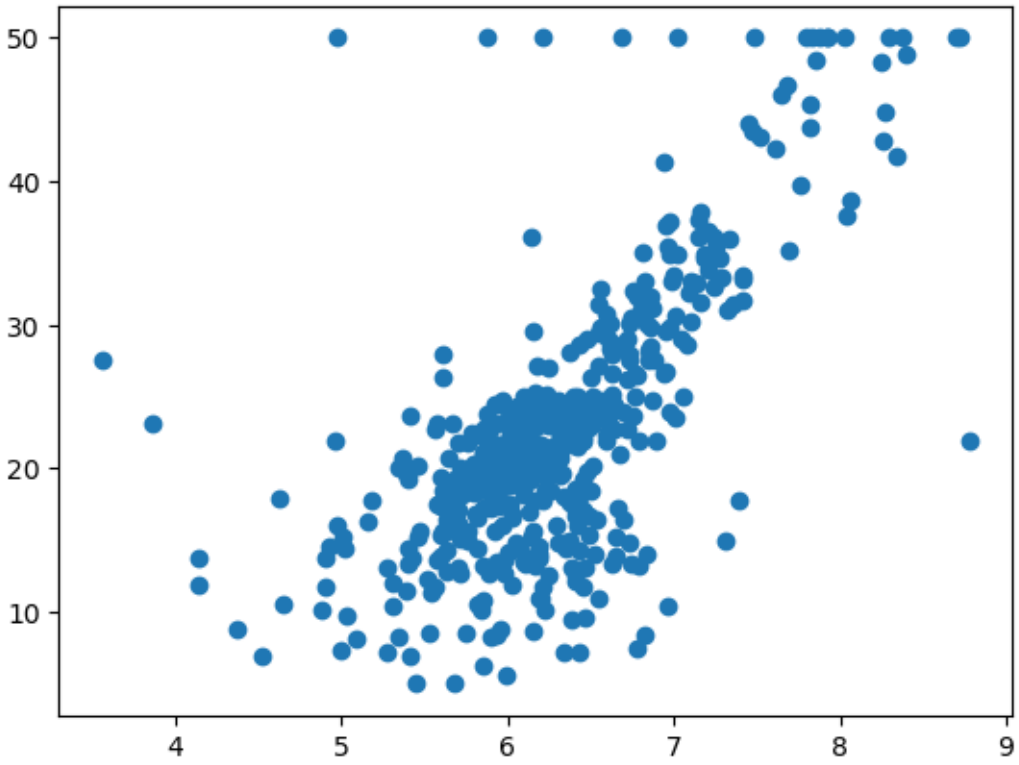
[47]:
```
plt.scatter(data.DIS,data.MEDV)
```

[47]: <matplotlib.collections.PathCollection at 0x1fab5d57090>



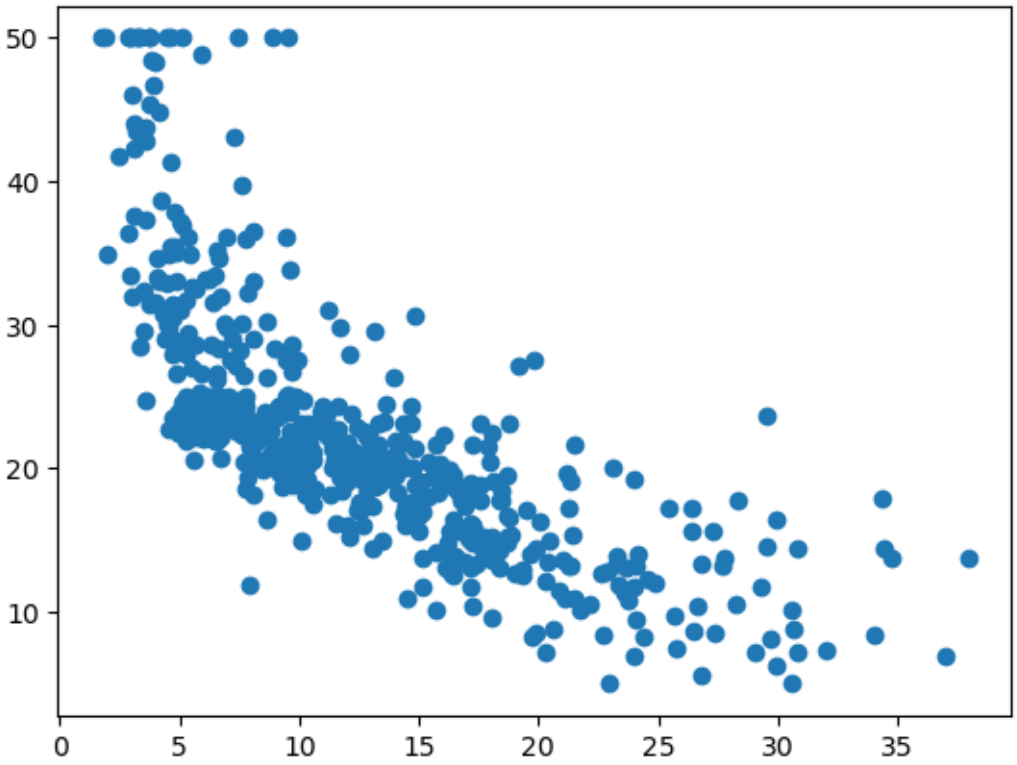
[48]:
```
plt.scatter(data.RM,data.MEDV)
```

[48]: <matplotlib.collections.PathCollection at 0x1fab5dc2750>



[36]:
```
plt.scatter(data.LSTAT,data.MEDV)
```

[36]: <matplotlib.collections.PathCollection at 0x1faafd88590>



3 question - 2

- Create a simple linear regression model that quantitatively relates 'MEDV' with 'RM'. (10 marks)
- Share the model performance metrics and print the full regression model with coefficients.
- Use the model to predict the price of the house for 'RM' = 7

[234]:
```
# data splitting
X_test1 = pd.DataFrame({"RM":[7]})
X = pd.DataFrame(data["RM"])
X["RM"] = X["RM"]**5
y = pd.DataFrame(data["MEDV"])

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.25,␣
 ↪random_state = 42)
# call and fit the model on training data
lm = LinearRegression()
lm = lm.fit(X_train, y_train)
# predict using test set
y_pred = lm.predict(X_test)
y_pred1 = lm.predict(X_test1)
```

```
# displaying coefficient, intercept, coefficint of regretion
display(lm.coef_, lm.intercept_, r2_score(y_test, y_pred),␣
 ↪mean_absolute_error(y_test, y_pred), mean_squared_error(y_test, y_pred))
display(y_pred1)
```

```
array([[0.00090513]])
```

```
array([12.47007994])
```

```
0.500584484104849
```

```
4.090370527811027
```

```
34.972642074769745
```

```
array([[12.47641581]])
```

[235]:
```
# data splitting
X_test1 = pd.DataFrame({"RM":[7]})
X = pd.DataFrame(data["RM"])
X["RM"] = X["RM"]**5
y = pd.DataFrame(data["MEDV"])

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.25,␣
 ↪random_state = 97)
# call and fit the model on training data
lm = LinearRegression()
lm = lm.fit(X_train, y_train)
# predict using test set
y_pred = lm.predict(X_test)
y_pred1 = lm.predict(X_test1)
# displaying coefficient, intercept, coefficint of regretion
display(lm.coef_, lm.intercept_, r2_score(y_test, y_pred),␣
 ↪mean_absolute_error(y_test, y_pred), mean_squared_error(y_test, y_pred))
display(y_pred1)
```

```
array([[0.00092229]])
```

```
array([12.23622933])
```

```
0.7791787150796949
```

```
3.6606879265719465
```

```
21.585025264797387
```

```
array([[12.24268538]])
```

[226]:
```
# # data splitting
# X_test1 = pd.DataFrame({"RM":[7]})
# X = pd.DataFrame(data["RM"])
# z=0.0
```

```
# t=0
# # X["RM"] = X["RM"]**5
# y = pd.DataFrame(data["MEDV"])
# for rs in range(0,550):
#     X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.
 ↪25, random_state = rs)
#     lm = LinearRegression()
#     lm = lm.fit(X_train, y_train)
#     y_pred = lm.predict(X_test)
#     y_pred1 = lm.predict(X_test1)
#     w=r2_score(y_test, y_pred)
#     if z < w:
#         t=rs
#         z=w
# t
# t=97
```

4 queston - 3

- Create a simple linear regression model that quantitatively relates 'MEDV' with 'DIS'. (10 marks)
- Share the model performance metrics and print the full regression model with coefficients.
- Use the model to predict the price of the house for 'DIS' = 15

[325]:
```
from math import log
# data splitting
X_test1 = pd.DataFrame({"DIS":[15]})
X = pd.DataFrame(data["DIS"])
# X["DIS"] = X.apply(lambda x: log(x["DIS"]**-30), axis = 1)
y = pd.DataFrame(data["MEDV"])
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.25,␣
 ↪random_state = 42)
# call and fit the model on training data
lm = LinearRegression()
lm = lm.fit(X_train, y_train)
# predict using test set
y_pred = lm.predict(X_test)
y_pred1 = lm.predict(X_test1)
# displaying coefficient, intercept, coefficint of regretion
display(lm.coef_, lm.intercept_, r2_score(y_test, y_pred),␣
 ↪mean_absolute_error(y_test, y_pred), mean_squared_error(y_test, y_pred))
display(y_pred1)
```

```
array([[1.09489544]])
```

```
array([18.78734243])
```

```
0.03916671813726291
```

```
6.123676402396705
67.2844102568222
array([[35.21077409]])
```

Name - Nikhil Khatri

SIC - 21BCSA92